

CE-CoLLM: Efficient and Adaptive Large Language Models Through Cloud-Edge Collaboration

Hongpeng Jin, Yanzhao Wu
Florida International University, Miami, FL, USA
{hjin008, yawu}@fiu.edu

Abstract—Large Language Models (LLMs) exhibit remarkable human-like predictive capabilities. However, it is challenging to deploy LLMs to provide efficient and adaptive inference services at the edge. This paper proposes a novel Cloud-Edge Collaboration framework for LLMs (CE-CoLLM) to tackle these challenges. First, we identify the transmission of LLM contextual data between the cloud and edge as a key performance bottleneck, which introduces substantial communication overhead that dominates overall inference latency and makes naïve cloud-edge collaboration for LLMs inefficient. Second, we introduce a suite of novel techniques, including a latency-aware early exit mechanism and efficient cloud context management, into CE-CoLLM, which collectively reduce communication overhead and preserve LLM inference accuracy. Third, we design two adaptive inference modes to accommodate diverse edge environments: (1) a low-latency standalone edge inference mode that enables reliable edge-side independent LLM inference even under unstable network conditions, and (2) a high-accuracy cloud-edge collaborative inference mode that adaptively leverages cloud resources to enhance prediction accuracy. Extensive experiments on multiple benchmark datasets demonstrate that CE-CoLLM reduces overall inference time by up to 13.81% and offloads over 84.53% of the computational workload from the cloud to the edge, compared to conventional cloud-based LLM deployment, without sacrificing prediction accuracy. The code is provided on GitHub at <https://github.com/mlsysx/CE-CoLLM>.

Index Terms—Large Language Model, LLM Deployment, Cloud-Edge Collaboration, Cloud Services, Adaptive LLM Inference, Edge AI.

I. INTRODUCTION

Large Language Models (LLMs) have demonstrated remarkable predictive capabilities, transforming diverse fields ranging from Natural Language Processing (NLP) to critical decision-making tasks [1]–[5]. There is a growing interest in extending the powerful predictive performance of LLMs directly to edge devices to gain benefits, such as low inference latency, enhanced privacy protection, and reliable inference independent of network connectivity [6]–[9]. However, realizing these objectives presents critical challenges. On the one hand, deploying full-scale LLMs directly on resource-constrained edge devices is often impractical due to their limited computational and memory resources [6], [10]–[12], typically requiring model compression or pruning techniques that compromise model accuracy [13]–[16]. On the other hand, relying solely on the conventional cloud-based LLM deployment [17]–[19], which utilizes substantial computational power in the cloud, introduces inherent performance issues, such as network-dependent communication latency and vulnerabilities under

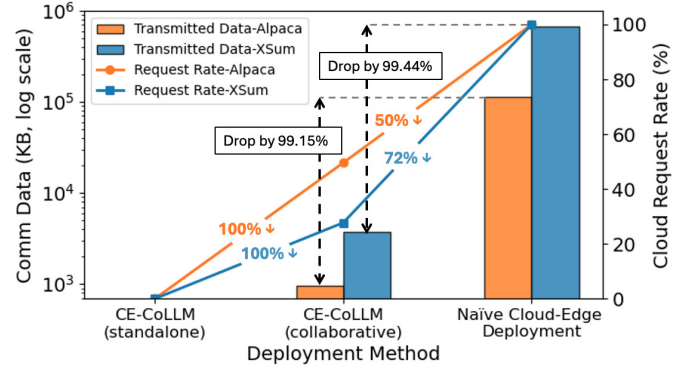


Fig. 1. Comparison of average transmitted data sizes per response and cloud request rates between CE-CoLLM and Naïve Cloud-Edge Deployment on Alpaca and XSum datasets. Bar plots represent the transmitted data size (log scale, KB), while solid lines indicate the cloud request rate (percentage). The results demonstrate that CE-CoLLM significantly reduces both communication overhead and reliance on cloud requests.

service or network interruptions [8], [9], [20]–[22]. These limitations highlight the pressing need to explore cloud-edge collaboration strategies, which hold the potential to harness the benefits of both cloud and edge computing to deliver efficient, adaptive, and reliable LLM-based services at the edge.

Several recent studies have made early attempts to enable cloud-edge collaboration for LLM deployment [8], [9], [23]. A popular approach is the hybrid deployment strategy, where a Small Language Model (SLM) is deployed at the edge to handle simple tokens, while complex tokens are offloaded to a powerful LLM in the cloud [8], [23]. While promising, such methods often result in redundant computation or suboptimal resource utilization. A more integrated alternative is to partition a single LLM across the cloud and edge to perform collaborative inference [9]. However, this split-model deployment introduces substantial communication overhead due to the iterative transmission of a large amount of contextual data (e.g., hidden states) between the cloud and edge for each token, as illustrated in Figure 2(b). Our study shows that frequent transmission of contextual data dominates overall inference latency, making the naïve cloud-edge collaborative LLM deployments slow and impractical for real-world applications.

To address these critical challenges, this paper introduces CE-CoLLM, a novel **Cloud-Edge Collaborative** framework for **LLMs**. Figure 1 shows an experimental comparison between CE-CoLLM and naïve cloud-edge deployment in terms of the average transmitted data size per response and cloud

Hongpeng Jin, Yanzhao Wu 佛罗里
达国际大学, 美国佛罗里达州迈阿密 {hjin
008, yawu}@fiu.edu

Abstract—大型语言模型 (LLM) 表现出卓越的类人预测能力。然而, 部署大语言模型以在边缘提供高效、自适应的推理服务具有挑战性。本文提出了一种新颖的大语言模型云边缘协作框架 (CE-CoLLM) 来应对这些挑战。首先, 我们将云端和边缘之间的 LLM 上下文数据传输确定为关键的性能瓶颈, 这会带来大量的通信开销, 主导整体推理延迟, 并使 LLM 的原生云边缘协作效率低下。其次, 我们在 CE-CoLLM 中引入了一套新技术, 包括延迟感知提前退出机制和高效的云上下文管理, 这些技术共同减少了通信开销并保持了 LLM 推理的准确性。第三, 我们设计了两种自适应推理模式来适应不同的边缘环境: (1) 低延迟独立边缘推理模式, 即使在不稳定的网络条件下也能实现可靠的边缘侧独立 LLM 推理; (2) 高精度云边协同推理模式, 自适应地利用云资源来提高预测精度。对多个基准数据集的大量实验表明, 与传统的基于云的 LLM 部署相比, CE-CoLLM 将整体推理时间缩短了高达 13.81%, 并将超过 84.53% 的计算工作负载从云端卸载到边缘, 且不牺牲预测精度。该代码在 GitHub 上提供: <https://github.com/mlsysx/CE-CoLLM>。

Index Terms—大语言模型、LLM部署、云边协作、云服务、自适应 LLM 推理、边缘人工智能。

一、简介

大语言模型 (LLM) 已经展示了显着的预测能力, 改变了从自然语言处理 (NLP) 到关键决策任务等不同领域[1]-[5]。人们越来越有兴趣将大语言模型强大的预测性能直接扩展到边缘设备, 以获得好处, 例如低推理延迟、增强的隐私保护以及独立于网络连接的可靠推理[6]-[9]。然而, 实现这些目标面临着严峻的挑战。一方面, 由于计算和内存资源有限, 直接在资源受限的边缘设备上部署全面的 LLM 通常是不切实际的[6]、[10]-[12], 通常需要模型压缩或修剪技术, 从而损害模型的准确性[13]-[16]。另一方面, 仅仅依靠传统的基于云的 LLM 部署[17]-[19], 利用云中的大量计算能力, 引入了固有的性能问题, 例如依赖于网络的通信延迟和漏洞

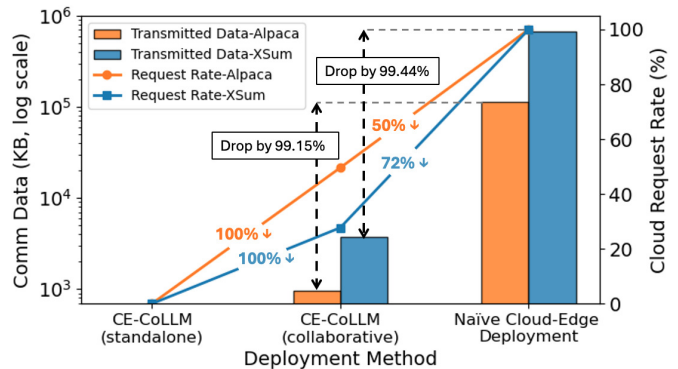


图 1. CE-CoLLM 与 Naive Cloud-Edge Deployment 在 Alpaca 和 XSum 数据集上的每次响应平均传输数据大小和云请求率的比较。条形图表示传输的数据大小 (对数刻度, KB), 而实线表示云请求率 (百分比)。结果表明, CE-CoLLM 显著降低了通信开销和对云请求的依赖。

服务或网络中断 [8]、[9]、[20]-[22]。这些限制凸显了探索云边缘协作策略的迫切需要, 该策略有可能利用云计算和边缘计算的优势, 在边缘提供高效、自适应且可靠的基于 LLM 的服务。

最近的几项研究已经做出了早期尝试, 以实现 LLM 部署的云边缘协作 [8]、[9]、[23]。一种流行的方法是混合部署策略, 其中在边缘部署小语言模型 (SLM) 来处理简单的令牌, 而复杂的令牌则卸载到云中强大的 LLM [8]、[23]。虽然很有前景, 但此类方法通常会导致冗余计算或资源利用率不佳。一种更集成的替代方案是将单个 LLM 跨云和边缘进行分区以执行协作推理 [9]。然而, 由于每个令牌在云端和边缘之间迭代传输大量上下文数据 (例如隐藏状态), 这种分割模型部署引入了大量的通信开销, 如图 2(b) 所示。我们的研究表明, 上下文数据的频繁传输在整体推理延迟中占主导地位, 使得简单的云边缘协作 LLM 部署速度缓慢且对于现实应用程序来说不切实际。

为了应对这些关键挑战, 本文介绍了 CE-CoLLM, 这是一种针对大语言模型的新型云边缘协作框架。图 1 显示了 CE-CoLLM 和原生云边缘部署在每个响应和云的平均传输数据大小方面的实验比较

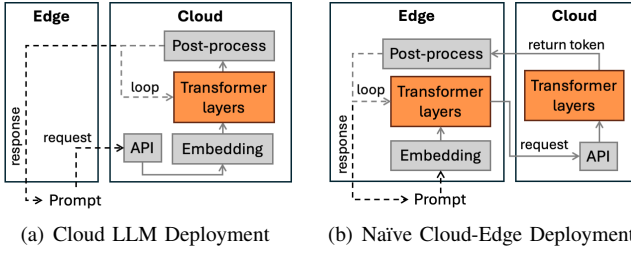


Fig. 2. Figure (a) shows the architecture of Cloud LLM Deployment. The edge devices access LLM inference services through API requests. Figure (b) is the architecture of Naïve Cloud-Edge Deployment. In this setup, hidden states are transmitted from the edge device to the cloud for each token inference. The cloud-side computation begins only after receiving these hidden states.

request rate on the Alpaca [24] and XSum [25] datasets. CE-CoLLM achieves significant reductions in both communication overhead and cloud request rates, compared to the naïve cloud-edge deployment. We make three original contributions. *First*, we conduct an empirical analysis to show the substantial communication overhead caused by transmitting contextual data in naïve cloud-edge deployment. *Second*, we propose the CE-CoLLM framework, which integrates a suite of novel components, including the latency-aware early exit mechanism and cloud context management, to effectively mitigate the communication bottleneck. CE-CoLLM offers two flexible inference modes to accommodate diverse edge environments: (1) low-latency standalone edge inference and (2) high-accuracy collaborative cloud-edge inference. *Third*, through comprehensive experiments on popular benchmark datasets, we demonstrate that CE-CoLLM significantly outperforms the naïve cloud-edge collaboration by drastically reducing communication overhead, leading to lower end-to-end inference latency while maintaining comparable prediction accuracy.

II. LLM DEPLOYMENT STRATEGY

In this section, we introduce three main strategies for deploying Large Language Models (LLMs): cloud deployment, edge deployment, and cloud-edge collaborative deployment.

Cloud LLM Deployment is a prevalent strategy that enables full-scale Large Language Model (LLM) inference by leveraging ample cloud computational resources. Popular LLM-based services, such as ChatGPT [17], [18] and Claude [19], are hosted in the cloud, providing edge-side users with remote access to these powerful LLMs, as illustrated in Figure 2(a). However, the cloud LLM deployment is highly dependent on network connectivity and service availability. This dependence incurs significant round-trip communication latency due to the transmission of prompts, responses and intermediate data, which makes the cloud-based LLM service vulnerable to disruptions caused by network instability or cloud-side load surges [16], [26], particularly hindering real-time applications at the edge.

Edge LLM Deployment executes LLMs directly on edge devices (e.g., mobile or IoT devices), bringing inference closer to end users. Representative frameworks include ChatRTX [27] and Ollama [28], which reduce network dependency and enable low-latency services through local inference at the

edge. However, deploying full-scale LLMs on edge devices is challenging due to their limited compute, memory, and storage resources. To accommodate these constraints, models are often compressed using techniques such as quantization or pruning [29]–[31]. While these methods effectively reduce model size, they can inadvertently compromise prediction accuracy or reduce the model’s ability to handle complex tasks [13], [15], [16], thereby limiting their utility for applications that demand high-quality LLM inference.

Cloud-Edge Collaborative LLM Deployment harnesses the extensive computational power of the cloud while utilizing the low-latency advantages of edge devices. LLM deployment across cloud and edge environments generally falls into two main categories. (1) *Separate Model Deployment*: A full-scale LLM is deployed in the cloud, while a Small Language Model (SLM) runs at the edge. Complex tasks are routed to the cloud to maintain prediction accuracy. However, this strategy can introduce redundant inference overhead if both the edge and cloud process the same input, resulting in suboptimal resource utilization. Furthermore, maintaining consistent context and state between the edge and cloud can be challenging [8], [23]. (2) *Split Model Deployment*: The LLM is split into edge and cloud partitions, with the initial layers (edge partition) deployed on the edge device and the remaining layers (cloud partition) hosted in the cloud. This split model deployment strategy aims to preserve the accuracy of a full-scale LLM. However, due to the auto-regressive nature of LLMs [32], where each token is generated based on preceding tokens, it incurs frequent transmission of substantial intermediate data between the edge and cloud for each generated token. As shown in Figure 1, the large volume of transmitted data introduces high communication latency, often dominating the overall inference latency.

To overcome the communication challenges associated with split model deployment, we propose CE-CoLLM, a novel cloud-edge collaboration framework for LLMs. CE-CoLLM enables adaptive inference task distribution between the cloud and edge, efficiently offloading computational workloads from the cloud to the edge to enhance resource utilization and reduce communication overhead. Furthermore, it provides an edge standalone mode for edge-side users, ensuring resilient and efficient inference at the edge even when the cloud network connection is interrupted.

III. CE-CoLLM OVERVIEW

In this section, we provide an overview of our proposed CE-CoLLM framework, designed to enable efficient and adaptive Cloud-Edge Collaboration for LLMs. CE-CoLLM consists of three key functioning components: (1) latency-aware early exit mechanisms, (2) asynchronous contextual data upload, and (3) efficient context management. As illustrated in Figure 4 (left), CE-CoLLM supports both standalone edge inference and adaptive cloud-edge collaborative inference modes. We observed that not all token generations require full LLM inference. Figure 3 shows the distribution of token generation confidence scores, which exhibits a long-tail pattern, with most

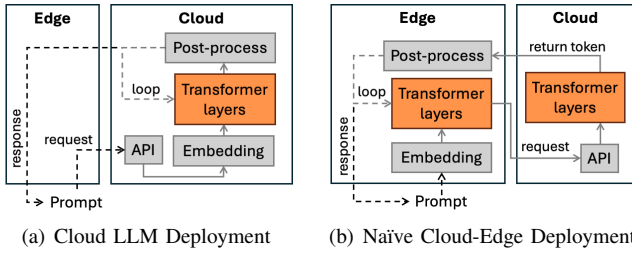


图2. 图(a)展示了Cloud LLM部署的架构。边缘设备通过API请求访问LLM推理服务。图(b)是Naïve Cloud-Edge Deployment的架构。在此设置中，每个令牌推理的隐藏状态都会从边缘设备传输到云端。云端计算仅在接收到这些隐藏状态后才开始。

Alpaca [24] 和 XSum [25] 数据集上的请求率。与简单的云边缘部署相比，CE-CoLLM 显着降低了通信开销和云请求率。我们做出了三项原创贡献。First,我们进行了实证分析，以显示在简单的云边缘部署中传输上下文数据所造成的大量通信开销。Second,我们提出了CE-CoLLM框架，它集成了一套新颖的组件，包括延迟感知提前退出机制和云上下文管理，以有效缓解通信瓶颈。CE-CoLLM提供两种灵活的推理模式来适应不同的边缘环境：（1）低延迟独立边缘推理和（2）高精度协作云边缘推理。Third,通过对流行基准数据集的全面实验，我们证明CE-CoLLM通过大幅减少通信开销，显着优于简单的云边缘协作，从而降低端到端推理延迟，同时保持可比较的预测精度。

二.大语言模型部署策略

在本节中，我们介绍部署大语言模型（LLM）的三种主要策略：云部署、边缘部署和云边协作部署。

云 LLM 部署是一种流行的策略，它通过利用充足的云计算资源来实现全面的大语言模型（LLM）推理。流行的基于 LLM 的服务，例如 ChatGPT [17]、[18] 和 Claude [19]，托管在云中，为边缘端用户提供对这些强大的 LLM 的远程访问，如图 2(a) 所示。然而，云LLM部署高度依赖于网络连接和服务可用性。由于提示、响应和中间数据的传输，这种依赖性会产生显着的往返通信延迟，这使得基于云的LLM服务容易受到网络不稳定或云端负载激增造成的中断[16], [26]，特别是阻碍边缘的实时应用程序。

边缘LLM部署直接在边缘设备（例如移动或物联网设备）上执行LLM，使推理更接近最终用户。代表性框架包括 ChatRTX [27] 和 Ollama [28]，它们减少网络依赖性并通过本地推理实现低延迟服务

边缘。然而，由于边缘设备的计算、内存和存储资源有限，在边缘设备上部署全面的 LLM 具有挑战性。为了适应这些约束，通常使用量化或修剪等技术来压缩模型[29]-[31]。虽然这些方法有效地减小了模型大小，但它们可能会无意中损害预测准确性或降低模型处理复杂任务的能力[13]、[15]、[16]，从而限制了它们在需要高质量 LLM 推理的应用程序中的实用性。

云边缘协作LLM部署利用云的广泛计算能力，同时利用边缘设备的低延迟优势。跨云和边缘环境的 LLM 部署通常分为两大类。（1）*Separate Model Deployment*: 全面的 LLM 部署在云端，而小语言模型（SLM）运行在边缘。复杂的任务被路由到云端以保持预测的准确性。然而，如果边缘和云端处理相同的输入，则此策略可能会引入冗余推理开销，从而导致资源利用率不佳。此外，维持边缘和云之间一致的上下文和状态可能具有挑战性[8]、[23]。（2）*Split Model Deployment*: LLM 分为边缘和云分区，初始层（边缘分区）部署在边缘设备上，其余层（云分区）托管在云端。这种分割模型部署策略旨在保持全面大语言模型的准确性。然而，由于 LLM [32] 的自回归性质，其中每个令牌都是基于先前的令牌生成的，因此会导致每个生成的令牌在边缘和云之间频繁传输大量中间数据。如图 1 所示，大量传输数据会带来较高的通信延迟，通常主导整体推理延迟。

为了克服与拆分模型部署相关的通信挑战，我们提出了 CE-CoLLM，这是一种针对大语言模型的新型云边缘协作框架。CE-CoLLM 支持云端和边缘之间的自适应推理任务分配，有效地将计算工作负载从云端卸载到边缘，以提高资源利用率并减少通信开销。此外，它还为边缘侧用户提供边缘独立模式，即使云网络连接中断，也能确保边缘的弹性和高效推理。

三. CE-COLM 概述

在本节中，我们概述了我们提出的 CE-CoLLM 框架，该框架旨在为大语言模型实现高效且自适应的云边缘协作。CE-CoLLM 包含三个关键功能组件：（1）延迟感知提前退出机制，（2）异步上下文数据上传，以及（3）高效上下文管理。如图 4（左）所示，CE-CoLLM 支持独立边缘推理和自适应云边协同推理模式。我们观察到，并非所有代币生成都需要完整的 LLM 推理。图 3 显示了令牌生成置信度得分的分布，呈现出长尾模式，其中大多数

generated tokens concentrated near the high confidence end. Specifically, 47.89% of tokens on the Alpaca dataset [24] and 68.26% on the XSum dataset [25] achieve confidence scores above 0.8 at an intermediate early exit layer. This indicates that a substantial portion of tokens can be generated with high confidence before reaching the final layer of an LLM, presenting opportunities to eliminate unnecessary computation in subsequent layers. This observation motivates the core design of CE-CoLLM with a latency-aware early exit mechanism that terminates inference at intermediate layers for high-confidence tokens. Moreover, the early exit mechanism provides a natural way to split an LLM into multiple partitions. The first several layers, along with one or more early exit points, constitute only a small fraction of the LLM and can be deployed on resource-constrained edge devices, while the remaining layers are deployed in the cloud. This design allows the edge device to efficiently process most tokens with high confidence, while more challenging cases (i.e., low-confidence tokens) are offloaded to the cloud to continue the inference, ensuring adaptive and efficient LLM deployment. We below describe the key components of CE-CoLLM in facilitating efficient and adaptive cloud-edge collaboration for LLMs.

A. Latency-aware Early Exit Mechanism

We incorporate a latency-aware early exit mechanism into CE-CoLLM to optimize the cloud-edge collaboration for LLMs. As shown in Figure 4 (left), this mechanism integrates multiple early exit points at intermediate layers of the LLM. The initial few layers, equipped with early exit points, constitute a lightweight edge partition suitable for deployment on resource-constrained edge devices, while the remaining layers form the cloud partition for execution in the cloud. During inference, each token is first processed through the edge partition and evaluated at each exit points. If the token's confidence score (*conf*) exceeds a predefined threshold (θ), it is generated directly at the early exit point locally without requiring cloud support. The confidence score is computed as:

$$\text{conf} = \max_i \frac{\exp(z_i)}{\sum_{j=1}^V \exp(z_j)},$$

where $\{z_1, z_2, \dots, z_V\}$ denote the output logits over a vocabulary of size V , and $i, j \in \{1, 2, \dots, V\}$. If the confidence score at the final edge-side early exit point still falls below the threshold θ , the token is then offloaded to the cloud, where the remaining inference is completed using the cloud partition.

This design enables two adaptive operational modes in CE-CoLLM: (1) *Edge Standalone Inference Mode (Low-Latency Mode)* is designed for scenarios with limited or unreliable network connectivity. In this mode, the edge-side LLM inference operates independently, generating tokens locally via edge-side early exit points without relying on the cloud. This ensures uninterrupted LLM-based services and low latency responses (see CE-CoLLM (standalone) performance in Table I). (2) *Adaptive Cloud-Edge Collaborative Inference Mode (High-Accuracy Mode)* serves as the default mode under stable network conditions with high prediction accuracy. In

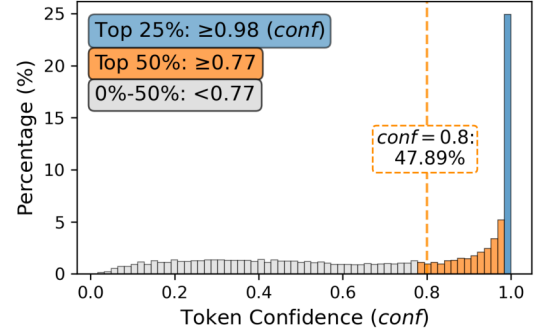


Fig. 3. The distribution of token confidence scores at an early exit layer for the Alpaca dataset, showing a long-tail pattern skewed toward high confidence scores, with 50% of tokens exceeding a confidence score of 0.77 and 25% exceeding 0.98.

this mode, if a token's confidence score remains below the predefined threshold θ at the edge, CE-CoLLM sends the token along with its necessary contextual information to the cloud for completing the remaining inference. This design enables the edge-side LLM inference to efficiently handle most token generations, while leveraging cloud support for a few challenging tokens to maintain prediction accuracy, as illustrated in Figure 4 (right). The threshold θ can be tuned to control the workload distribution between the edge and cloud. By default, we recommend setting $\theta = 0.8$ to achieve a balanced trade-off between inference latency and prediction accuracy.

B. Asynchronous Contextual Data Upload

Figure 1 illustrates that transmitting substantial contextual data (e.g., hidden states) when invoking cloud support for low-confidence tokens incurs significant communication overhead. To address this issue, CE-CoLLM introduces an asynchronous mechanism for uploading contextual data. Since most tokens can be confidently generated at the edge, CE-CoLLM leverages this opportunity to decouple data transmission from synchronous inference process by uploading contextual data in parallel with ongoing edge-side inference. Specifically, once edge-side LLM inference reaches a predefined model layer and triggers an upload operation (automatically or upon meeting certain conditions, such as a confidence threshold), it proactively transmits the current contextual data to the cloud-side context manager. This design ensures that, if cloud support is ultimately required, the necessary contextual information is already available or nearly transferred, allowing cloud-side inference to begin with minimal data transfer delay. By overlapping edge-side computation with data transmission, CE-CoLLM effectively masks a significant portion of the communication overhead, significantly reducing overall inference latency. Furthermore, to further optimize the communication efficiency, we convert the data transmitted from `float32` to `float16`, thereby halving the data size without compromising model accuracy, as validated in Section IV-B.

生成的代币集中在高置信度端附近。具体来说，Alpaca 数据集 [24] 上的 47.89% 和 XSum 数据集 [25] 上的 68.26% 的标记在中间早期退出层的置信度得分高于 0.8。这表明在到达 LLM 的最后一层之前，可以以高置信度生成很大一部分令牌，从而为消除后续层中不必要的计算提供了机会。这一观察结果激发了 CE-CoLLM 的核心设计，它具有延迟感知的提前退出机制，可终止中间层对高置信度令牌的推理。此外，提前退出机制提供了一种将 LLM 拆分为多个分区的自然方式。前几层以及一个或多个早期退出点仅构成 LLM 的一小部分，可以部署在资源受限的边缘设备上，而其余层则部署在云中。这种设计允许边缘设备以高置信度高效处理大多数令牌，同时将更具挑战性的案例（即低置信度令牌）卸载到云端以继续推理，从而确保自适应且高效的 LLM 部署。下面我们将描述 CE-CoLLM 在促进大语言模型高效、自适应云边缘协作方面的关键组件。

A. Latency-aware Early Exit Mechanism

我们将延迟感知提前退出机制纳入 CE-CoLLM，以优化大语言模型的云边缘协作。如图4（左）所示，该机制在 LLM 的中间层集成了多个早期退出点。最初的几层配备了早期退出点，构成了适合部署在资源受限的边缘设备上的轻量级边缘分区，而其余层则形成了云分区，以便在云中执行。在推理过程中，每个标记首先通过边缘分区进行处理，并在每个出口点进行评估。如果令牌的置信度分数 ($conf$) 超过预定义阈值 (θ)，则直接在早期退出点本地生成，无需云支持。置信度得分计算如下：

$$conf = \max_i \frac{\exp(z_i)}{\sum_{j=1}^V \exp(z_j)},$$

其中 $\{z_1, z_2, \dots, z_V\}$ 表示大小为 V 的词汇表的输出 logits， $i, j \in \{1, 2, \dots, V\}$ 。如果最终边缘侧提前退出点的置信度分数仍然低于阈值 θ ，则令牌将被卸载到云，其中使用云分区完成剩余的推理。

此设计在 CE-CoLLM 中实现了两种自适应操作模式：(1) *Edge Standalone Inference Mode (Low-Latency Mode)* 专为网络连接有限或不可靠的场景而设计。在这种模式下，边缘侧 LLM 推理独立运行，通过边缘侧早期退出点在本地产生成令牌，而不依赖于云。这确保了基于 LLM 的服务不间断和低延迟响应（参见表 I 中的 CE-CoLLM（独立）性能）。(2) 在网络稳定的情况下，*Adaptive Cloud-Edge Collaborative Inference Mode (High-Accuracy Mode)* 作为默认模式，预测精度较高。在

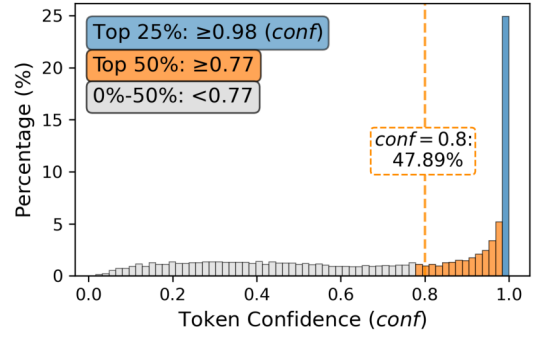


图 3. Alpaca 数据集早期退出层的 token 置信度得分分布，显示出偏向高置信度得分的长尾模式，其中 50% 的 token 置信度得分超过 0.77，25% 的 token 超过 0.98。

在此模式下，如果令牌的置信度分数在边缘保持低于预定义阈值 θ ，CE-CoLLM 会将令牌及其必要的上下文信息发送到云以完成剩余的推理。这种设计使边缘侧 LLM 推理能够有效地处理大多数令牌生成，同时利用云对一些具有挑战性的令牌的支持来保持预测准确性，如图 4（右）所示。可以调整阈值 θ 以控制边缘和云之间的工作负载分配。默认情况下，我们建议设置 $\theta = 0.8$ 以实现推理延迟和预测精度之间的平衡权衡。

B. Asynchronous Contextual Data Upload

图 1 说明，在调用低置信度令牌的云支持时传输大量上下文数据（例如，隐藏状态）会产生大量通信开销。为了解决这个问题，CE-CoLLM 引入了异步机制来上传上下文数据。由于大多数令牌可以在边缘放心地生成，CE-CoLLM 利用这个机会通过与正在进行的边缘端推理并行上传上下文数据来将数据传输与同步推理过程解耦。具体来说，一旦边缘侧 LLM 推理到达预定义的模型层并触发上传操作（自动或满足某些条件，例如置信度阈值），它就会主动将当前上下文数据传输到云端上下文管理器。这种设计确保，如果最终需要云支持，必要的上下文信息已经可用或即将传输，从而允许云端推理以最小的数据传输延迟开始。通过将边缘计算与数据传输重叠，CE-CoLLM 有效地屏蔽了很大一部分通信开销，从而显着减少了整体推理延迟。此外，为了进一步优化通信效率，我们将传输的数据从 float32 转换为 float16，从而在不影响模型精度的情况下将数据大小减半，如第 IV-B 节中所验证的。

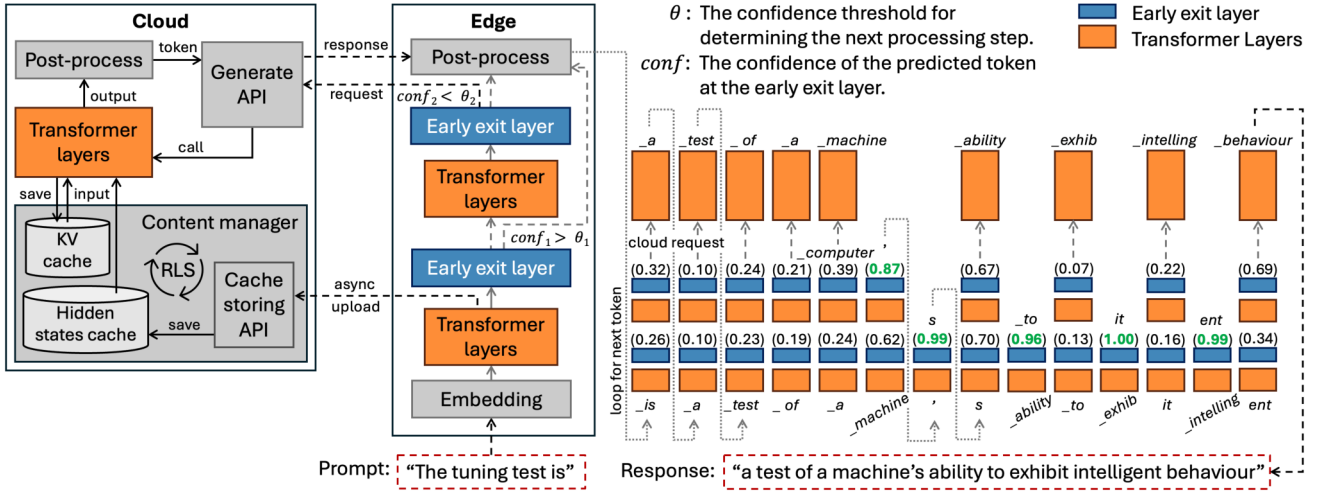


Fig. 4. CE-CoLLM architecture overview and workflow: (left) deployment of CE-CoLLM across the edge and cloud with two early-exit points at the edge; (right) end-to-end cloud-edge inference workflow, where tokens with confidence scores ≥ 0.8 are generated locally via an early exit point, while these falling below the threshold are offloaded to the cloud for continued inference.

C. Cloud Support for LLM Inference

The cloud-side LLM partition comprises the remaining LLM layers not deployed at the edge, which is designed to continue inference for low-confidence tokens offloaded by the edge side. The cloud-side LLM inference incorporates three key mechanisms to support efficient and scalable cloud-edge collaboration. (1) *Context Manager* serves as the core component that manages the contextual data for each edge client. It stores asynchronously uploaded contextual data (e.g., hidden states) from edge clients to ensure that all necessary context is available when cloud-side LLM inference is requested. The context manager also maintains Key-Value (KV) caches generated during cloud-side inference, preserving them across the token generation sequence to avoid redundant computations on previously processed tokens. Unused contextual data and key-value (KV) caches are automatically cleared upon session completion or after a specified period of inactivity, ensuring efficient resource utilization. (2) *Single-Token Response* is employed by the cloud server to return only one token for each cloud inference request, rather than the entire prediction probability vector. This design effectively reduces communication overhead while still supporting subsequent token generation at the edge. (3) *Dual APIs* are provided by the cloud server to support separate execution paths for contextual data uploads and inference processing, allowing both operations to proceed in parallel. These mechanisms enable the cloud server to efficiently support edge clients in continuing LLM inference for low-confidence tokens while reducing communication and computation costs. This design supports adaptive, efficient, and scalable cloud-edge collaboration for delivering LLM-based services to multiple edge clients.

D. Cloud-Edge Collaboration Workflow

The end-to-end inference workflow of CE-CoLLM is illustrated in Figure 4 (right, with default $\theta=0.8$). For each input prompt, inference begins on the edge device, where the edge-side LLM partition processes the input and evaluates the

confidence score at each early exit point. Once the confidence score meets or exceeds the threshold ($conf \geq \theta$), the token is generated locally without invoking cloud support. Meanwhile, CE-CoLLM asynchronously updates contextual data to the cloud in parallel with ongoing edge-side inference. This proactive transmission prepares the cloud server for potential cloud support requests. If a token's confidence score remains below the threshold θ at the final edge-side exit point, the edge client requests cloud support. The cloud-side Context Manager retrieves the uploaded contextual data and stored KV cache to allow the cloud-side LLM partition to resume and complete the remaining LLM inference. The generated token is then returned to the edge client, which appends it to the token sequence and proceeds with generating the next token. This process repeats until the generation is completed.

In summary, CE-CoLLM effectively addresses the communication bottleneck in cloud-edge collaborative LLM deployment through three coordinated mechanisms: (1) latency-aware early exit mechanisms, (2) asynchronous contextual data upload, and (3) efficient context management. This design enables CE-CoLLM to handle most token predictions at the edge, significantly offloading computational load from the cloud to the edge, thereby ensuring efficient resource utilization while maintaining comparable prediction accuracy.

IV. EXPERIMENTAL ANALYSIS

We conduct experimental analysis primarily from two perspectives to evaluate CE-CoLLM: (1) assessing runtime performance improvements, including faster inference and reduced communication overhead, compared to existing deployment strategies; and (2) verifying that CE-CoLLM preserves prediction accuracy on par with the widely used cloud-based LLM deployment. This section focuses on analyzing the performance of four key LLM deployment strategies. (1) *Cloud LLM Deployment*: A conventional strategy introduced in Section II and illustrated in Figure 2(a), serving as our accuracy baseline. (2) *Naïve Cloud-Edge Deployment*: A baseline

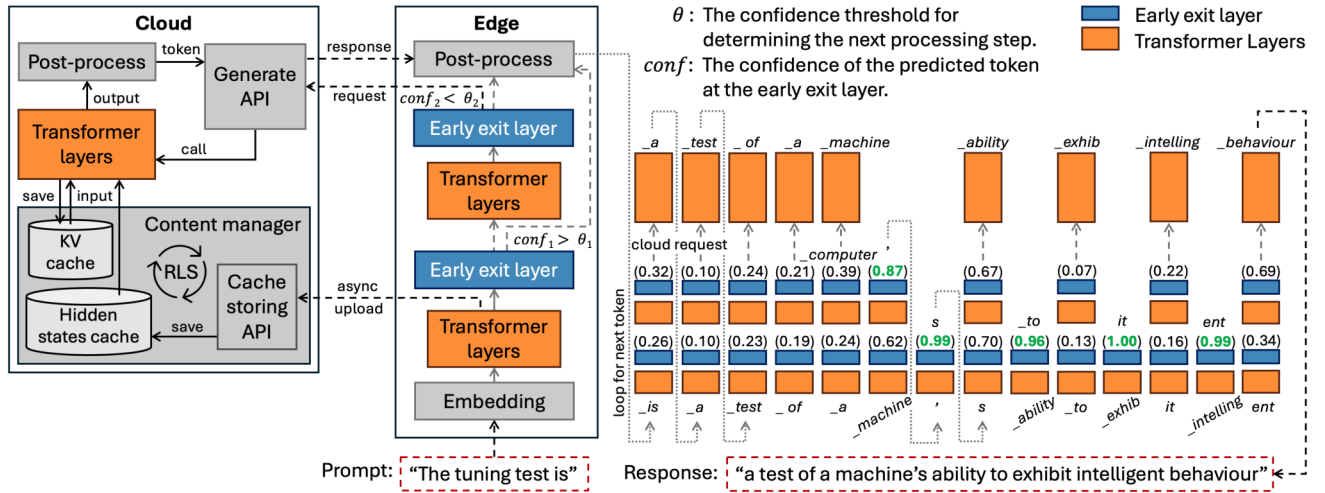


图 4. CE-CoLLM 架构概述和工作流程：（左）跨边缘和云部署 CE-CoLLM，在边缘有两个早期退出点；（右）端到端云边缘推理工作流程，其中置信度得分 ≥ 0.8 的令牌是通过早期退出点在本地生成的，而低于阈值的令牌则被卸载到云端以继续推理。

C. Cloud Support for LLM Inference

云端 LLM 分区会损害未部署在边缘的剩余 LLM 层，其目的是继续对边缘侧卸载的低置信度令牌进行推理。云端 LLM 推理融合了三个关键机制来支持高效、可扩展的云边协作。（1）*Context Manager* 作为核心组件，管理每个边缘客户端的上下文数据。它存储从边缘客户端异步上传的上下文数据（例如隐藏状态），以确保在请求云端 LLM 推理时所有必要的上下文可用。上下文管理器还维护在云端推理期间生成的键值（KV）缓存，在整个令牌生成序列中保留它们，以避免对先前处理的令牌进行冗余计算。未使用的上下文数据和键值（KV）缓存会在会话完成时或指定的不活动时间后自动清除，从而确保资源的高效利用。（2）云服务器使用 *Single-Token Response* 为每个云推理请求仅返回一个令牌，而不是整个预测概率向量。这种设计有效地减少了通信开销，同时仍然支持后续在边缘生成令牌。（3）*Dual APIs* 由云服务器提供，支持上下文数据上传和推理处理的单独执行路径，允许两个操作并行进行。这些机制使云服务器能够有效地支持边缘客户端对低置信度令牌进行持续的 LLM 推理，同时降低通信和计算成本。该设计支持自适应、高效且可扩展的云边缘协作，以便向多个边缘客户端提供基于 LLM 的服务。

D. Cloud-Edge Collaboration Workflow

CE-CoLLM 的端到端推理工作流程如图 4 所示（右，默认 $\theta=0.8$ ）。对于每个输入提示，推理在边缘设备上开始，其中边缘侧 LLM 分区处理输入并评估

每个早期退出点的置信度得分。一旦置信度分数达到或超过阈值 ($conf \geq \theta$)，就会在本地生成令牌，而无需调用云支持。与此同时，CE-CoLLM 与正在进行的边缘推理并行地将上下文数据异步更新到云端。这种主动传输使云服务器为潜在的云支持请求做好准备。如果令牌的置信度分数在最终边缘侧出口点仍然低于阈值 θ ，则边缘客户端请求云支持。云端上下文管理器检索上传的上下文数据和存储的 KV 缓存，以允许云端 LLM 分区恢复并完成剩余的 LLM 推理。然后，生成的令牌返回到边缘客户端，边缘客户端将其附加到令牌序列并继续生成下一个令牌。重复此过程直至生成完成。

总之，CE-CoLLM 通过三种协调机制有效解决了云边缘协作 LLM 部署中的通信瓶颈：（1）延迟感知提前退出机制，（2）异步上下文数据上传，以及（3）高效上下文管理。这种设计使 CE-CoLLM 能够在边缘处理大多数令牌预测，从而显著地将计算负载从云端卸载到边缘，从而确保高效的资源利用，同时保持可比的预测精度。

四. 实验分析

我们主要从两个角度进行实验分析来评估 CE-CoLLM：

（1）评估与现有部署策略相比运行时性能的改进，包括更快的推理和减少的通信开销；（2）验证 CE-CoLLM 的预测准确性与广泛使用的基于云的 LLM 部署相当。本节重点分析四种关键 LLM 部署策略的绩效。（1）

Cloud LLM Deployment：第二节中介绍的传统策略，如图 2(a) 所示，作为我们的准确性基线。（2）

Naïve Cloud-Edge Deployment：基线

TABLE I
PERFORMANCE COMPARISON ACROSS DIFFERENT LLM DEPLOYMENT STRATEGIES

Dataset	Deployment Strategy	Total Cost (s)	Cloud Cost (s)	Edge Cost (s)	Comm Cost (s)
Alpaca	Cloud LLM Deployment	370.166 $\pm$ 1.692	369.799 $\pm$ 1.682	0	0.367 $\pm$ 0.013
	Naïve Cloud-Edge Deployment	3371.761 $\pm$ 34.023	253.358 $\pm$ 1.695	241.428 $\pm$ 8.832	2876.975 $\pm$ 24.815
	CE-CoLLM (standalone)	201.574 $\pm$ 0.278	0	201.574 $\pm$ 0.278	0
	CE-CoLLM (collaborative)	<u>319.057</u> $\pm$ 0.438	112.723 $\pm$ 0.211	192.204 $\pm$ 0.437	14.130 $\pm$ 0.181
XSum	Cloud LLM Deployment	392.466 $\pm$ 7.075	392.012 $\pm$ 7.128	0	0.454 $\pm$ 0.106
	Naïve Cloud-Edge Deployment	19108.662 $\pm$ 211.580	271.661 $\pm$ 5.321	411.340 $\pm$ 2.520	18425.868 $\pm$ 207.460
	CE-CoLLM (standalone)	221.388 $\pm$ 0.865	0	221.388 $\pm$ 0.865	0
	CE-CoLLM (collaborative)	<u>375.999</u> $\pm$ 0.720	60.652 $\pm$ 0.133	258.383 $\pm$ 0.595	56.963 $\pm$ 0.137

strategy that splits an LLM for separate cloud and edge deployments as shown in Figure 2(b). (3) *CE-CoLLM (standalone)*: The standalone mode of our proposed CE-CoLLM framework, operating independently at the edge. (4) *CE-CoLLM (collaborative)*: The adaptive cloud-edge collaborative inference mode of our CE-CoLLM framework, using the optimal confidence threshold of $\theta = 0.8$.

Experimental Setup: We evaluate CE-CoLLM using a 7B LLaMA model [33] equipped with two early exit points [34], which is trained to enable adaptive LLM inference by terminating the inference process at an early exit once the confidence exceeds a pre-defined threshold (θ). The cloud and edge partitions are deployed separately on a cloud server and an edge device. The cloud server is equipped with a single NVIDIA A100 GPU, which is consistent across all deployment strategies to ensure a fair comparison. We mainly use two representative datasets, Alpaca [24] and XSum [25], to evaluate the LLM inference performance. To validate inference accuracy across different downstream tasks, we employ BoolQ [35] and QuAC [36] for question answering, IMDB [37] for sentiment analysis, and XSum for summarization.

Evaluation Metrics: We evaluate these LLM deployment strategies using two categories of metrics: (1) *runtime performance metrics*, including total inference time cost, edge computation time cost, cloud computation time cost, and communication time cost, and (2) *accuracy metrics*, including Exact Match (EM) [38] for BoolQ and IMDB, F1 score for QuAC, and ROUGE-L [39] for XSum.

A. Runtime Performance Analysis

We first evaluate runtime performance on a single edge device equipped with an NVIDIA A100 GPU, using 100 randomly selected samples from Alpaca (with relatively short prompts ranging from 13 to 43 tokens) and XSum (with relatively long prompts ranging from 200 to 500 tokens). LLM generates up to 100 tokens for each prompt. The main experiments are repeated five times, and results are reported as mean and standard deviation (mean $\pm$ std). Table I presents the cumulative time cost for 100 prompts, broken down into cloud computation, edge computation, and communication time costs. We highlight three interesting observations.

First, the naïve cloud-edge deployment suffers from prohibitively high communication costs, making it impractical

for real-world applications. Processing a single case takes an average of 33.72 seconds on the Alpaca dataset and 191.09 seconds on the XSum dataset, primarily due to the excessive communication overhead of sending all requests to the cloud. In contrast, as illustrated in Figure 1, CE-CoLLM significantly reduces cloud requests to 49.58% on Alpaca and 27.73% on XSum, while dramatically decreasing the average data transmitted per request, from 112,128 KB to 956.62 KB on Alpaca (a 99.15% reduction) and from 673,520.03 KB to 3763.61 KB on XSum (a 99.44% reduction). *Second*, CE-CoLLM significantly outperforms the conventional cloud LLM deployment and delivers enhanced efficiency and inference speed. As shown in Table I, CE-CoLLM achieves much faster inference of 319 seconds on Alpaca and 376 seconds on XSum, compared to 370 seconds and 392 seconds, for cloud LLM deployment, corresponding to 13.81% and 4.19% performance improvements, respectively. Furthermore, CE-CoLLM effectively offloads computation from the cloud to the edge, reducing the cloud execution time costs to 113 seconds on Alpaca and 61 seconds on XSum, achieving reductions of 69.52% and 84.53%, respectively, compared to cloud LLM deployment. This substantial shift of workload to the edge enables the cloud to significantly improve computational efficiency and potentially support more concurrent edge clients using the same resources. *Third*, CE-CoLLM offers flexible deployment modes to accommodate varying edge environments and network conditions. The standalone mode delivers the fastest inference, with 201.57 seconds on Alpaca and 221.39 seconds on XSum, while completely eliminating cloud dependency, making it suitable for adverse scenarios with unstable network connectivity. Meanwhile, the adaptive cloud-edge collaborative inference mode substantially reduces the cloud computational load by 69.52% on Alpaca and 84.53% on XSum, while leveraging the cloud to maintain inference accuracy (see Table II).

B. Impact on LLM Inference Accuracy

We then assess the impact of CE-CoLLM on prediction accuracy in comparison to conventional cloud-based LLM deployment across three representative downstream tasks: (1) BoolQ and QuAC for question answering, (2) IMDB for sentiment analysis, and (3) XSum for summarization. Table II presents the experimental results. Overall, we observe

表 I 不同 LLM 部署策略的绩效比较

Dataset	Deployment Strategy	Total Cost (s)	Cloud Cost (s)	Edge Cost (s)	Comm Cost (s)
Alpaca	Cloud LLM Deployment	370.166 $\pm$ 1.692	369.799 $\pm$ 1.682	0	0.367 $\pm$ 0.013
	Naïve Cloud-Edge Deployment	3371.761 $\pm$ 34.023	253.358 $\pm$ 1.695	241.428 $\pm$ 8.832	2876.975 $\pm$ 24.815
	CE-CoLLM (standalone)	201.574 $\pm$ 0.278	0	201.574 $\pm$ 0.278	0
	CE-CoLLM (collaborative)	<u>319.057</u> $\pm$ 0.438	112.723 $\pm$ 0.211	192.204 $\pm$ 0.437	14.130 $\pm$ 0.181
XSum	Cloud LLM Deployment	392.466 $\pm$ 7.075	392.012 $\pm$ 7.128	0	0.454 $\pm$ 0.106
	Naïve Cloud-Edge Deployment	19108.662 $\pm$ 211.580	271.661 $\pm$ 5.321	411.340 $\pm$ 2.520	18425.868 $\pm$ 207.460
	CE-CoLLM (standalone)	221.388 $\pm$ 0.865	0	221.388 $\pm$ 0.865	0
	CE-CoLLM (collaborative)	<u>375.999</u> $\pm$ 0.720	60.652 $\pm$ 0.133	258.383 $\pm$ 0.595	56.963 $\pm$ 0.137

将 LLM 拆分为单独的云和边缘部署的策略，如图 2(b) 所示。(3) *CE-CoLLM (standalone)*: 我们提出的 CE-CoLLM 框架的独立模式，在边缘独立运行。(4)

CE-CoLLM (collaborative): 我们的 CE-CoLLM 框架的自适应云边协作推理模式，使用 $\theta = 0.8$ 的最佳置信阈值。

实验设置：我们使用配备两个早期退出点[34]的 7B LLaMA 模型[33]评估 CE-CoLLM，该模型经过训练，一旦置信度超过预定义阈值 (θ)，就通过在早期退出处终止推理过程来实现自适应 LLM 推理。云和边缘分区分别部署在云服务器和边缘设备上。云服务器配备了单个 NVIDIA A100 GPU，在所有部署策略中都是一致的，以确保公平比较。我们主要使用两个代表性数据集 Alpaca [24] 和 XSum [25] 来评估 LLM 推理性能。为了验证不同下游任务的推理准确性，我们使用 BoolQ [35] 和 QuAC [36] 进行问答，使用 IMDB [37] 进行情感分析，使用 XSum 进行摘要。

评估指标：我们使用两类指标评估这些 LLM 部署策略：(1) *runtime performance metrics*，包括总推理时间成本、边缘计算时间成本、云计算时间成本和通信时间成本，以及 (2) *accuracy metrics*，包括 BoolQ 和 IMDB 的精确匹配 (EM) [38]、QuAC 的 F1 分数和 XSum 的 ROUGE-L [39]。

A. Runtime Performance Analysis

我们首先在配备 NVIDIA A100 GPU 的单边缘设备上评估运行时性能，使用来自 Alpaca（相对较短的提示，范围从 13 到 43 个令牌）和 XSum（相对较长的提示，范围从 200 到 500 个令牌）随机选择的 100 个样本。LLM 为每个提示生成最多 100 个令牌。主要实验重复五次，结果以平均值和标准差 (mean $\pm$ std) 形式报告。表 I 列出了 100 个提示的累积时间成本，分为云计算、边缘计算和通信时间成本。我们强调三个有趣的观察结果。

First，简单的云边缘部署面临着过高的通信成本，使其不切实际

对于现实世界的应用程序。在 Alpaca 数据集上处理单个案例平均需要 33.72 秒，在 XSum 数据集上处理单个案例平均需要 191.09 秒，这主要是由于将所有请求发送到云端的通信开销过大。相比之下，如图 1 所示，CE-CoLLM 在 Alpaca 上将云请求显着减少到 49.58%，在 XSum 上减少了 27.73%，同时显着减少了每个请求传输的平均数据，在 Alpaca 上从 112,128 KB 减少到 956.62 KB（减少了 99.15%），在 Alpaca 上从 673,520.03 KB 减少到 3763.61 KB XSum（减少 99.44%）。*Second*，CE-CoLLM 显着优于传统的云 LLM 部署，并提供更高的效率和推理速度。如表一所示，CE-CoLLM 在 Alpaca 上实现了 319 秒的推理，在 XSum 上实现了 376 秒的推理，而云 LLM 部署的推理速度为 370 秒和 392 秒，分别相当于 13.81% 和 4.19% 的性能提升。此外，CE-CoLLM 有效地将计算从云端卸载到边缘，将 Alpaca 上的云执行时间成本降低到 113 秒，XSum 上的云执行时间成本降低到 61 秒，与云 LLM 部署相比，分别减少了 69.52% 和 84.53%。工作负载向边缘的这种实质性转移使云能够显着提高计算效率，并有可能支持使用相同资源的更多并发边缘客户端。*Third*，CE-CoLLM 提供灵活的部署模式，以适应不同的边缘环境和网络条件。单机模式推理速度最快，Alpaca 为 201.57 秒，XSum 为 221.39 秒，同时完全消除了对云的依赖，适合网络连接不稳定的恶劣场景。同时，自适应云边协同推理模式在 Alpaca 上大幅降低云计算负载 69.52%，在 XSum 上降低 84.53%，同时利用云保持推理精度（见表二）。

B. Impact on LLM Inference Accuracy

然后，我们在三个代表性下游任务中评估 CE-CoLLM 与传统的基于云的 LLM 部署相比对预测准确性的影响：(1) 用于问答的 BoolQ 和 QuAC，(2) 用于情感分析的 IMDB，以及 (3) 用于摘要的 XSum。表二给出了实验结果。总体而言，我们观察到

TABLE II
ACCURACY COMPARISON BETWEEN CE-CoLLM (STANDALONE AND COLLABORATIVE MODES) AND CLOUD LLM DEPLOYMENT

Task Type	Dataset	CE-CoLLM		Cloud LLM Deployment
		Standalone	Collaborative	
Question	BoolQ (EM)	0.4900	0.6580	0.6460
Analysis	QuAC (F1)	0.2740	0.2890	0.2910
Sentiment Analysis	IMDB (EM)	0.6840	0.7240	0.7240
Summarization	XSum (Rouge-L)	0.1917	0.2250	0.2275

that CE-CoLLM (collaborative) in the cloud-edge collaborative inference mode achieves comparable prediction accuracy to the cloud-based LLM deployment. Specifically, for the question answering task, CE-CoLLM delivers an accuracy of 0.658 on BoolQ and 0.289 on QuAC, closely matching the accuracy achieved by cloud LLM deployment of 0.646 and 0.291, respectively. For sentiment analysis on IMDB, CE-CoLLM produces the same accuracy as the cloud-based LLM deployment. In the summarization task on XSum, CE-CoLLM achieves an accuracy of 0.225, which is comparable to the cloud LLM deployment with an accuracy of 0.2275. These experimental results further validate that CE-CoLLM maintains inference accuracy on par with cloud-based LLM deployment. In the standalone edge inference mode, it incurs only a small accuracy drop while delivering reliable, low-latency inference at the edge without relying on cloud support.

Overall, CE-CoLLM achieves significant performance improvements across multiple dimensions over existing LLM deployment strategies. *First*, compared to naïve cloud-edge deployment, CE-CoLLM effectively mitigates the communication bottleneck by dramatically reducing both the rate of cloud requests and the volume of transmitted data, making cloud-edge collaboration efficient for real-world applications. *Second*, CE-CoLLM significantly outperforms conventional cloud-based LLM deployment in terms of inference speed while effectively offloading computation from the cloud to the edge and preserving inference accuracy, demonstrating the efficacy of the proposed CE-CoLLM framework. *Third*, CE-CoLLM provides a standalone edge inference mode that ensures uninterrupted services even under unstable network conditions, thereby enhancing overall system resilience compared to cloud-dependent LLM deployment strategies. These advantages position CE-CoLLM as a highly practical solution for delivering efficient and adaptive LLM-based services across diverse edge environments.

V. RELATED WORK

The related studies can be summarized into three broad categories: (1) Cloud-Edge Collaborative ML, (2) Early Exit Mechanisms, and (3) Large Language Models.

Cloud-Edge Collaborative ML harnesses the high computational power of cloud servers and the low-latency benefits of edge devices to efficiently distribute Machine Learning (ML) workloads across both cloud and edge. While most existing efforts [40], [41] focus on deploying conventional Deep Neural Networks (DNNs), cloud-edge collaborative LLM deployment

faces unique challenges due to the high communication overhead caused by the inherent iterative inference loops in LLMs. A few recent studies have made early attempts to explore cloud-edge collaboration for LLMs. For instance, [8] deploys a Small Language Model (SLM) at the edge while offloading challenging tokens to a cloud-based LLM, and [9] splits an LLM using dynamic programming to optimize throughput and latency. However, these existing cloud-edge LLM deployment approaches still suffer from high communication overhead and heavy dependence on the cloud, leaving them vulnerable to network disruptions.

Early Exit Mechanisms provide an efficient way to dynamically adjust the depth of deep neural network inference, enabling faster and adaptive input processing by allowing the DNN to terminate inference at intermediate layers [42]–[45]. Several studies have incorporated early exit mechanisms into computer vision models [45], [46] and language models [34], [47]–[49], including recent LLMs [50]–[54]. However, few studies investigate efficient frameworks that leverage early exit mechanisms to enable adaptive cloud-edge collaboration for deploying LLMs.

Large Language Models (LLMs) have achieved remarkable success in various domains [55]–[57], represented by the GPT series [58]–[60], ChatGPT [17], and open-source models, such as LLaMA [33], [61], [62] and Mistral [63]–[65]. Despite their impressive predictive capabilities, the high computational demands pose critical challenges for delivering LLM-based services at the edge [10], [66]–[68]. To tackle these challenges, we propose an efficient and adaptive cloud-edge collaboration framework to empower LLM-based services at the edge.

VI. CONCLUSION

This paper introduces CE-CoLLM, a novel cloud-edge collaboration framework that accelerates LLM inference at the edge with optional cloud support. *First*, we identify and quantify the substantial communication overhead in naïve cloud-edge LLM deployment, where contextual data transmission dominates inference latency. *Second*, we propose CE-CoLLM to effectively address this critical communication bottleneck through efficient and adaptive cloud-edge collaboration for LLM deployment. *Third*, comprehensive experiments on multiple benchmark datasets demonstrate that CE-CoLLM significantly reduces the overall LLM inference latency, cloud computation load, and communication overhead while maintaining comparable prediction accuracy. Overall, CE-CoLLM provides an effective and reliable solution for delivering efficient and adaptive LLM-based services at the edge.

ACKNOWLEDGMENT

The authors acknowledge the National Artificial Intelligence Research Resource (NAIRR) Pilot (NAIRR240244), OpenAI, and Amazon Web Services for partially contributing to this research result. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of funding agencies and companies mentioned above.

表 II CE-COLL M（独立模式和协作模式）与云 LLM 部署之间的精度比较

Task Type	Dataset	CE-CoLLM		Cloud LLM Deployment
		Standalone	Collaborative	
Question	BoolQ (EM)	0.4900	0.6580	0.6460
Analysis	QuAC (F1)	0.2740	0.2890	0.2910
Sentiment Analysis	IMDB (EM)	0.6840	0.7240	0.7240
Summarization	XSum (Rouge-L)	0.1917	0.2250	0.2275

云边协同推理模式下的CE-CoLLM（协作）实现了与基于云的LLM部署相当的预测精度。具体来说，对于问答任务，CE-CoLLM 在 BoolQ 上的准确度为 0.658，在 QuAC 上的准确度为 0.289，与云 LLM 部署分别达到的 0.646 和 0.291 的准确度非常接近。对于 IMDB 上的情感分析，CE-CoLLM 的准确性与基于云的 LLM 部署相同。在 XSum 上的摘要任务中，CE-CoLLM 达到了 0.225 的准确率，与云 LLM 部署的 0.2275 的准确率相当。这些实验结果进一步验证了 CE-CoLLM 保持了与基于云的 LLM 部署相同的推理精度。在独立边缘推理模式下，精度仅略有下降，同时在边缘提供可靠、低延迟的推理，无需依赖云支持。

总体而言，与现有的 LLM 部署策略相比，CE-CoLLM 在多个维度上实现了显著的性能改进。与简单的云边缘部署相比，CE-CoLLM 通过大幅降低云请求速率和传输数据量来有效缓解通信瓶颈，使云边缘协作对于实际应用程序而言更加高效。*Second*, CE-CoLLM 在推理速度方面显著优于传统的基于云的 LLM 部署，同时有效地将计算从云端卸载到边缘并保持推理准确性，证明了所提出的 CE-CoLLM 框架的有效性。*Third*, CE-CoLLM 提供独立的边缘推理模式，即使在不稳定的网络条件下也能确保服务不间断，从而与依赖云的 LLM 部署策略相比增强了整体系统的弹性。这些优势使 CE-CoLLM 成为一种高度实用的解决方案，可在不同的边缘环境中提供高效且自适应的基于 LLM 的服务。

五、相关工作

相关研究可概括为三大类：（1）云边协作机器学习，（2）提前退出机制，（3）大语言模型。

云边缘协作机器学习利用云服务器的高计算能力和边缘设备的低延迟优势，跨云和边缘有效地分配机器学习（ML）工作负载。虽然大多数现有工作 [40]、[41] 侧重于部署传统的深度神经网络（DNN），但云边缘协作 LLM 部署

由于大语言模型固有的迭代推理循环造成的高通信开销，面临着独特的挑战。最近的一些研究已经做出了探索法学硕士云边缘协作的早期尝试。例如，[8] 在边缘部署小语言模型（SLM），同时将具有挑战性的代币卸载到基于云的 LLM，并且 [9] 使用动态编程拆分 LLM 以优化吞吐量和延迟。然而，这些现有的云边缘 LLM 部署方法仍然面临着高通信开销和对云的严重依赖，使它们容易受到网络中断的影响。

提前退出机制提供了一种动态调整深度神经网络推理深度的有效方法，通过允许 DNN 在中间层终止推理来实现更快的自适应输入处理[42]-[45]。一些研究已将早期退出机制纳入计算机视觉模型[45]、[46]和语言模型[34]、[47]-[49]，包括最近的大语言模型[50]-[54]。然而，很少有研究调查利用早期退出机制来实现自适应云边缘协作以部署法学硕士的有效框架。

大语言模型（LLM）在各个领域取得了显著的成功[55]-[57]，以GPT系列[58]-[60]、ChatGPT[17]为代表，以及开源模型，如LLaMA[33]、[61]、[62]和Mistral[63]-[65]。尽管它们的预测能力令人印象深刻，但高计算需求对在边缘提供基于 LLM 的服务提出了严峻的挑战 [10]、[66]-[68]。为了应对这些挑战，我们提出了一个高效且自适应的云边缘协作框架，以增强基于 LLM 的边缘服务。

六. 结论

本文介绍了 CE-CoLLM，这是一种新颖的云边缘协作框架，可通过可选的云支持加速边缘的 LLM 推理。*First*, 我们识别并量化了原生云边缘 LLM 部署中的大量通信开销，其中上下文数据传输主导推理延迟。*Second*, 我们建议 CE-CoLLM 通过 LLM 部署的高效和自适应云边缘协作来有效解决这一关键的通信瓶颈。*Third*, 对多个基准数据集的综合实验表明，CE-CoLLM 显著降低了整体 LLM 推理延迟、云计算负载和通信开销，同时保持了相当的预测精度。总体而言，CE-CoLLM 提供了一种有效且可靠的解决方案，可在边缘提供高效且自适应的基于 LLM 的服务。

致谢

作者感谢国家人工智能研究资源 (NAIRR) Pilot (NAIRR 240244)、OpenAI 和 Amazon Web Services 对本研究结果的部分贡献。本材料中表达的任何意见、发现、结论或建议均为作者的观点，并不一定反映上述资助机构和公司的观点。

REFERENCES

- [1] T. Teubner, C. M. Flath, C. Weinhardt, W. van der Aalst, and O. Hinz, "Welcome to the era of chatgpt et al. the prospects of large language models," *Business & Information Systems Engineering*, vol. 65, no. 2, pp. 95–101, 2023. [Online]. Available: <https://doi.org/10.1007/s12599-023-00795-x>
- [2] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Barnes, and A. Mian, "A comprehensive overview of large language models," *arXiv preprint arXiv:2307.06435*, 2023.
- [3] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, W. Ye, Y. Zhang, Y. Chang, P. S. Yu, Q. Yang, and X. Xie, "A survey on evaluation of large language models," *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 3, Mar. 2024. [Online]. Available: <https://doi.org/10.1145/3641289>
- [4] B. C. Das, M. H. Amini, and Y. Wu, "Security and privacy challenges of large language models: A survey," *ACM Comput. Surv.*, vol. 57, no. 6, Feb. 2025. [Online]. Available: <https://doi.org/10.1145/3712001>
- [5] R. Rangaraj, J. Shi, A. Shirali, R. Paudel, Y. Wu, and G. Narasimhan, "How Effective are Large Time Series Models in Hydrology? A Study on Water Level Forecasting in Everglades," *arXiv e-prints*, p. arXiv:2505.01415, May 2025.
- [6] Y. Wu, L. Liu, and R. Kompella, "Parallel detection for efficient video analytics at the edge," in *2021 IEEE Third International Conference on Cognitive Machine Intelligence (CogMI)*, 2021, pp. 01–10.
- [7] S. V. Ganesh, Y. Wu, G. Liu, R. Kompella, and L. Liu, "Amplifying object tracking performance on edge devices," in *2023 IEEE 5th International Conference on Cognitive Machine Intelligence (CogMI)*, 2023, pp. 83–92.
- [8] Z. Hao, H. Jiang, S. Jiang, J. Ren, and T. Cao, "Hybrid slm and llm for edge-cloud collaborative inference," in *Proceedings of the Workshop on Edge and Mobile Foundation Models*, ser. EdgeFM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 36–41. [Online]. Available: <https://doi.org/10.1145/3662006.3662067>
- [9] M. Zhang, X. Shen, J. Cao, Z. Cui, and S. Jiang, "Edgeshard: Efficient llm inference via collaborative edge computing," *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13 119–13 131, 2025.
- [10] Y. Wu, L. Liu, C. Pu, W. Cao, S. Sahin, W. Wei, and Q. Zhang, "A comparative measurement study of deep learning as a service framework," *IEEE Transactions on Services Computing*, vol. 15, no. 1, pp. 551–566, 2022.
- [11] M. Xu, D. Cai, W. Yin, S. Wang, X. Jin, and X. Liu, "Resource-efficient algorithms and systems of foundation models: A survey," *ACM Comput. Surv.*, vol. 57, no. 5, Jan. 2025.
- [12] G. Qu, Q. Chen, W. Wei, Z. Lin, X. Chen, and K. Huang, "Mobile edge intelligence for large language models: A contemporary survey," *IEEE Communications Surveys & Tutorials*, pp. 1–1, 2025.
- [13] W. Huang, Y. Liu, H. Qin, Y. Li, S. Zhang, X. Liu, M. Magno, and X. Qi, "Billm: pushing the limit of post-training quantization for llms," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.
- [14] A. Kundu, Y. C. F. Lim, A. Chew, L. Wynter, P. Chong, and R. Lee, "Efficiently distilling LLMs for edge applications," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, Y. Yang, A. Davani, A. Sil, and A. Kumar, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 52–62.
- [15] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei, "The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits," *arXiv e-prints*, p. arXiv:2402.17764, Feb. 2024.
- [16] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," in *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. D. Sa, Eds., vol. 6, 2024, pp. 87–100.
- [17] L. Ouyang, J. Wu, X. Jiang, and et al., "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 27 730–27 744.
- [18] OpenAI, J. Achiam, S. Adler, S. Agarwal, and et al., "GPT-4 Technical Report," *arXiv e-prints*, p. arXiv:2303.08774, Mar. 2023.
- [19] Anthropic, "Claude: An ai assistant model," <https://www.anthropic.com/claude>, 2024, accessed: 2024-10-29.
- [20] V. Ganatra, A. Parayil, S. Ghosh, Y. Kang, M. Ma, C. Bansal, S. Nath, and J. Mace, "Detection is better than cure: A cloud incidents perspective," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 1891–1902.
- [21] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen, J. Zeng, S. Ghosh, X. Zhang, C. Zhang, Q. Lin, S. Rajmohan, D. Zhang, and T. Xu, "Automatic root cause analysis via large language models for cloud incidents," in *Proceedings of the Nineteenth European Conference on Computer Systems*, ser. EuroSys '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 674–688.
- [22] Y. Ding, C. Niu, F. Wu, S. Tang, C. Lyu, and G. Chen, "Enhancing on-device llm inference with historical cloud-based llm interactions," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 597–608.
- [23] Apple, "Apple intelligence: AI for the rest of us," <https://www.apple.com/apple-intelligence/>, 2024, accessed: 2024-10-29.
- [24] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, "Stanford alpaca: An instruction-following llama model," https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [25] S. Narayan, S. B. Cohen, and M. Lapata, "Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, Eds. Brussels, Belgium: Association for Computational Linguistics, Oct.-Nov. 2018, pp. 1797–1807.
- [26] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, H. Jin, T. Chen, and Z. Jia, "Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems," *arXiv e-prints*, p. arXiv:2312.15234, Dec. 2023.
- [27] NVIDIA, "Chatrtx: Bringing generative ai to consumers with nvidia ai on RTX," <https://www.nvidia.com/en-us/ai-on-rtx/chatrtx/>, 2024, accessed: 2024-10-29.
- [28] Ollama, "Ollama: Open large language model api," <https://ollama.com/>, 2024, accessed: 2024-10-29.
- [29] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, "A simple and effective pruning approach for large language models," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=PxoFut3dWW>
- [30] T. Dettmers, R. Svirschevski, V. Egiazarian, D. Kuznedelev, E. Frantar, S. Ashkboos, A. Borzunov, T. Hoefler, and D. Alistarh, "Spqr: A sparse-quantized representation for near-lossless LLM weight compression," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=Q1u25ahSuy>
- [31] E. Frantar and D. Alistarh, "Sparsegpt: massive language models can be accurately pruned in one-shot," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023.
- [32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017.
- [33] H. Touvron, L. Martin, K. Stone, and et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," *arXiv e-prints*, p. arXiv:2307.09288, Jul. 2023.
- [34] Y. Chen, X. Pan, Y. Li, B. Ding, and J. Zhou, "Ee-llm: large-scale training and inference of early-exit large language models with 3d parallelism," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.
- [35] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "BoolQ: Exploring the surprising difficulty of natural yes/no questions," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 2924–2936.

- [1] T. Teubner, C. M. Flath, C. Weinhardt, W. van der Aalst 和 O. Hinz, “欢迎来到 chatgpt 等人的时代, 大语言模型的前景”, *Business & Information Systems Engineering*, 卷. 65, 没有. 2, 第 95–101 页, 2023 年. [在线]. 可用: <https://doi.org/10.1007/s12599-023-00795-x> [2] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Barnes 和 A. Mian, “大语言模型的综合概述”, *arXiv preprint arXiv:2307.06435*, 2023 年. [3] Y. Chang, X. Wang, J. 王, 吴, 杨, 朱, 陈, , 易, 王, 王, 叶, 张, 张, 余, 杨, 谢, “大语言模型评估综述”, *ACM Trans. Intell. Syst. Technol.*, 卷. 15, 没有. 2024 年 3 月 3 日. [在线]. 可用: <https://doi.org/10.1145/3641289> [4] B. C. Das, M. H. Amini 和 Y. Wu, “大语言模型的安全和隐私挑战: 一项调查”, *ACM Comput. Surv.*, 卷. 57, 没有. 2025 年 2 月 6 日. [在线]. 网址: <https://doi.org/10.1145/3712001> [5] R. Rangaraj, J. Shi, A. Shiri, R. Paudel, Y. Wu 和 G. Narasimhan, “大型时间序列模型在水文学中的效果如何? 大沼泽地水位预测研究”, *arXiv e-prints*, 第 14 页. arXiv:2505.01415, 2025 年 5 月. [6] Y. Wu, L. Liu 和 R. Kompella, “边缘高效视频分析的并行检测”, *2021 IEEE Third International Conference on Cognitive Machine Intelligence (CogMI)*, 2021 年, 第 01-10 页. [7] S. V. Ganesh, Y. Wu, G. Liu, R. Kompella 和 L. Liu, “增强边缘设备上的对象跟踪性能”, *2023 IEEE 5th International Conference on Cognitive Machine Intelligence (CogMI)*, 2023 年, 第 83-92 页. [8] Z. Hao, H. Jiang, S. Jiang, J. Ren 和 T. Cao, “用于边云协作推理的混合 slm 和 llm”, 见 *Proceedings of the Workshop on Edge and Mobile Foundation Models*, ser. EdgeFM '24. 美国纽约州纽约市: 计算机协会, 2024 年, 第 14 页. 36-41. [在线的]. 可用: <https://doi.org/10.1145/3662006.3662067> [9] M. Zhang, X. Shen, J. Cao, Z. Cui 和 S. Jiang, “Edgeshard: 通过协作边缘计算进行高效 llm 推理”, *IEEE Internet of Things Journal*, 卷. 12, 没有. 10, 第 13 119–13 131, 2025. [10] Y. Wu, L. Liu, C. Pu, W. Cao, S. Sahin, W. Wei, 和 Q. Zhang, “深度学习作为服务框架的比较测量研究”, *IEEE Transactions on Services Computing*, 卷. 15, 没有. 1, 第 551–5 66 页, 2022 年. [11] M. Xu, D. Cai, W. Yin, S. Wang, X. Jin 和 X. Li u, “基础模型的资源高效算法和系统: 一项调查”, *ACM Comput. Surv.*, 卷. 57, 没有. 2025 年 1 月 5 日. [12] G. Qu, Q. Chen, W. Wei, Z. Lin, X. Chen 和 K. Huang, “大语言模型的移动边缘智能: 当代调查”, *IEEE Communications Surveys & Tutorials*, 第 1-1 页, 2025 年. [13] W. Huang, Y. Liu, H. Qin, Y. Li, S. Zhang, X. Liu, M. Magno 和 X. Qi, “Billm: 突破 llms 训练后量化的极限”, *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024 年. [14] A. Kundu, Y. C. F. Lim, A. Che w, L. Wynter, P. Chong 和 R. Lee, “为边缘应用有效提炼大语言模型”, 载于 *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, Y. Yang, A. Davani, A. Sil 和 A. Kumar, 编辑. 墨西哥墨西哥城: 计算语言学协会, 2024 年 6 月, 第 52-62 页. [15] S. M a, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xu 和 F. Wei, “1 位大语言模型时代: 所有大语言模型均采用 1.58 位”, *arXiv e-prints*, 第 15 页. arXiv:2402.17764, 2024 年 2 月. [16] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. 陈, W.-C. Wang, G. Xiao, X. Dang, C. Gan 和 S. Han, “Awq: 设备上 llm 压缩和加速的激活感知权重量化”, 见 *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko 和 C. D. Sa, 编辑, 卷. 6, 2024 年, 第 87-100 页. [17] L. Ouyang, J. Wu, X. Jiang 等人, “训练语言模型以遵循人类反馈的指令”, 载于 *Advances in Neural Information Processing Systems*, S. Kojima, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho 和 A. Oh, 编辑, 卷. 35. Curran Associates, Inc., 2022 年, 第 27 730–27 744 页. [18] OpenAI, J. Achiam, S. Adler, S. Agarwal 等人, “GPT-4 技术报告”, *arXiv e-prints*, 第 35 页. arXiv: 2303.08774, 2023 年 3 月.
- [19] Anthropic, “克劳德: 人工智能助理模型”, <https://www.anthropic.com/claude>, 2024 年, 访问时间: 2024-10-29. [20] V. Ganatra, A. Parayil, S. Ghosh, Y. Kang, M. Ma, C. Bansal, S. Nath 和 J. Mace, “检测胜于治疗: 云事件视角”, 参见 *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2023. 美国纽约州纽约市: 计算机协会, 2023 年, 第 1 页. 1891–1902. [21] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gau, L. Shi, Y. Cao, X. Gau, H. Fan, M. Wen, J. Zeng, S. Ghosh, X. Zhang, C. Zhang, Q. Lin, S. Rajmohan, D. Zhu 和 T. Xu, “通过大语言模型对云事件进行自动根本原因分析”, 载于 *Proceedings of the Nineteenth European Conference on Computer Systems*, ser. EuroSys '24. 美国纽约州纽约市: 计算机协会, 2024 年, 第 14 页. 674–688. [22] Y. Ding, C. Niu, F. Wu, S. Tang, C. Lyu 和 G. Chen, “通过基于云的历史 llm 交互增强设备上 llm 推理”, 在 *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD' 24. 美国纽约州纽约市: 计算机协会, 2024 年, 第 14 页. 597–608. [23] 苹果, “苹果智能: 为我们其他人服务的人工智能”. <https://www.apple.com/apple-intelligence/>, 2024 年, 访问时间: 2024 年 10 月 29 日. [24] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang 和 T. B. Hashimoto, “斯坦福羊驼: 遵循指令的美洲驼模型”, https://github.com/tatsu-lab/stanford_alpaca, 2023. [25] S. Narayan, S. B. Cohen 和 M. Lapata, “不要给我细节, 只给我摘要! 用于极端摘要的主题感知卷积神经网络”, 见 *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, E. Riloff, D. Jiang, J. Hockenmaier 和 J. Tsujii, 编辑. 比利时布鲁塞尔: 计算语言学协会, 10 月至 11 月 2018 年, 第 1797–1807 页. [26] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, H. Jin, T. Chen 和 Z. Jia, “迈向高效的生成式大型语言模型服务: 从算法到系统的调查”, *arXiv e-prints*, 第 17 页. arXiv:2312.15234, 2023 年 12 月. [27] NVIDIA, “Chatrtx: 通过 RTX 上的 nvidia ai 为消费者带来生成式人工智能”, <https://www.nvidia.com/en-us/ai-on-rtx/ch>, 访问时间: 2024-10-29. [28] Ollama, “Ollama: 开放大语言模型 api”, <https://ollama.com/>, 2024 年, 访问时间: 2024 年. Sun, Z. Liu, A. Bair 和 J. Z. Kolter, “一种简单有效的大语言模型修剪方法”, 参见 *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [在线]. 可用: <https://openreview.net/forum?id=PxoFut3dWW> [30] T. Dettmers, R. Svirschevski, V. Egiazarian, D. Kuznedelev, E. Frantar, S. Ashkboos, A. B. Orzunov, T. Hoefler 和 D. Alistarh, “Spqr: 近无损 LLM 权重压缩的稀疏量化表示”, 参见 *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [在线]. 可用: <https://openreview.net/forum?id=Q1u25ahSuy> [31] E. Frantar 和 D. Alistarh, “Sparsegpt: 可以一次性准确地修剪大规模语言模型”, *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023. [32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser 和 I. Polosukhin, “注意力就是你所需要的”, 见 *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan 和 R. Garnett, 编辑, 卷. 30. Curran Associates, Inc., 2017 年. [33] H. Touvron, L. Martin, K. Stone 等人, “Llama 2: 开放基础和微调聊天模型”, *arXiv e-prints*, 第 34 页. arXiv:2307.09288, 2023 年 7 月. [34] Y. Chen, X. Pan, Y. Li, B. Ding 和 J. Zhou, “Ee-llm: 具有 3d 并行性的早期退出大型语言模型的大规模训练和推理”, 在 *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024 年. [35] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins 和 K. Toutanova, “BoolQ: 探索自然语言问题的惊人难度”, 载于 *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran 和 T. Solorio, 编辑. 明尼苏达州明尼阿波利斯: 计算语言学协会, 2019 年 6 月, 第 2924-2936 页.

- [36] E. Choi, H. He, M. Iyyer, M. Yatskar, W.-t. Yih, Y. Choi, P. Liang, and L. Zettlemoyer, "QuAC: Question answering in context," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, Eds. Brussels, Belgium: Association for Computational Linguistics, Oct.-Nov. 2018, pp. 2174–2184. [Online]. Available: <https://aclanthology.org/D18-1241/>
- [37] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, D. Lin, Y. Matsumoto, and R. Mihalcea, Eds. Portland, Oregon, USA: Association for Computational Linguistics, Jun. 2011, pp. 142–150. [Online]. Available: <https://aclanthology.org/P11-1015/>
- [38] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, J. Su, K. Duh, and X. Carreras, Eds. Austin, Texas: Association for Computational Linguistics, Nov. 2016, pp. 2383–2392. [Online]. Available: <https://aclanthology.org/D16-1264>
- [39] C.-Y. Lin and F. J. Och, "Automatic evaluation of machine translation quality using longest common subsequence and skip-bigram statistics," in *Proceedings of the 42nd Annual Meeting of the Association for Computational Linguistics (ACL-04)*, Barcelona, Spain, Jul. 2004, pp. 605–612. [Online]. Available: <https://aclanthology.org/P04-1077>
- [40] Y. Gao, W. Wang, D. Wang, H. Wang, and Z. Zhang, "Cloud-edge inference under communication constraints: Data quantization and early exit," in *2022 International Symposium on Wireless Communication Systems (ISWCS)*, 2022, pp. 1–6.
- [41] D. Xu, X. He, T. Su, and Z. Wang, "A Survey on Deep Neural Network Partition over Cloud, Edge and End Devices," *arXiv e-prints*, p. arXiv:2304.10020, Apr. 2023.
- [42] A. Graves, "Adaptive Computation Time for Recurrent Neural Networks," *arXiv e-prints*, p. arXiv:1603.08983, Mar. 2016.
- [43] R. Schwartz, G. Stanovsky, S. Swayamdipta, J. Dodge, and N. A. Smith, "The right tool for the job: Matching model and instance complexities," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds. Online: Association for Computational Linguistics, Jul. 2020, pp. 6640–6651. [Online]. Available: <https://aclanthology.org/2020.acl-main.593>
- [44] T. Schuster, A. Fisch, T. Jaakkola, and R. Barzilay, "Consistent accelerated inference via confident adaptive transformers," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 4962–4979. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.406>
- [45] F. Ilhan, K.-H. Chow, S. Hu, T. Huang, S. Tekin, W. Wei, Y. Wu, M. Lee, R. Kompella, H. Latapie, G. Liu, and L. Liu, "Adaptive deep neural network inference optimization with eenet," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, January 2024, pp. 1373–1382.
- [46] S. Teerapittayanon, B. McDanel, and H. Kung, "Branchynet: Fast inference via early exiting from deep neural networks," in *2016 23rd International Conference on Pattern Recognition (ICPR)*, 2016, pp. 2464–2469.
- [47] W. Liu, P. Zhou, Z. Wang, Z. Zhao, H. Deng, and Q. Ju, "FastBERT: a self-distilling BERT with adaptive inference time," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds. Online: Association for Computational Linguistics, Jul. 2020, pp. 6035–6044. [Online]. Available: <https://aclanthology.org/2020.acl-main.537/>
- [48] L. Hou, Z. Huang, L. Shang, X. Jiang, X. Chen, and Q. Liu, "Dynabert: dynamic bert with adaptive width and depth," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS '20. Red Hook, NY, USA: Curran Associates Inc., 2020.
- [49] W. Zhou, C. Xu, T. Ge, J. McAuley, K. Xu, and F. Wei, "Bert loses patience: Fast and robust inference with early exit," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 18 330–18 341.
- [50] L. Del Corro, A. Del Giorno, S. Agarwal, B. Yu, A. Awadallah, and S. Mukherjee, "SkipDecode: Autoregressive Skip Decoding with Batching and Caching for Efficient LLM Inference," *arXiv e-prints*, p. arXiv:2307.02628, Jul. 2023.
- [51] S. Bae, J. Ko, H. Song, and S.-Y. Yun, "Fast and robust early-exiting framework for autoregressive language models with synchronized parallel decoding," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 5910–5924. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.362>
- [52] S. Tang, Y. Wang, Z. Kong, T. Zhang, Y. Li, C. Ding, Y. Wang, Y. Liang, and D. Xu, "You need multiple exiting: Dynamic early exiting for accelerating unified vision language model," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 10 781–10 791.
- [53] N. Varshney, A. Chatterjee, M. Parmar, and C. Baral, "Investigating acceleration of LLaMA inference by enabling intermediate layer decoding via instruction tuning with 'LITE'," in *Findings of the Association for Computational Linguistics: NAACL 2024*, K. Duh, H. Gomez, and S. Bethard, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 3656–3677. [Online]. Available: <https://aclanthology.org/2024.findings-naacl.232>
- [54] X. Pan, Y. Chen, Y. Li, B. Ding, and J. Zhou, "EE-Tuning: An Economical yet Scalable Solution for Tuning Early-Exit Large Language Models," p. arXiv:2402.00518, Feb. 2024.
- [55] J. Kaddour, J. Harris, M. Mozes, H. Bradley, R. Raileanu, and R. McHardy, "Challenges and Applications of Large Language Models," *arXiv e-prints*, p. arXiv:2307.10169, Jul. 2023.
- [56] S. Hu, T. Huang, K.-H. Chow, W. Wei, Y. Wu, and L. Liu, "Zipzap: Efficient training of language models for large-scale fraud detection on blockchain," in *Proceedings of the ACM Web Conference 2024*, ser. WWW '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 2807–2816. [Online]. Available: <https://doi.org/10.1145/3589334.3645352>
- [57] J. Shi, A. Shirali, B. Jin, S. Zhou, W. Hu, R. Rangaraj, S. Wang, J. Han, Z. Wang, U. Lall, Y. Wu, L. Bobadilla, and G. Narasimhan, "Deep Learning and Foundation Models for Weather Prediction: A Survey," *arXiv e-prints*, p. arXiv:2501.06907, Jan. 2025.
- [58] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," 2018.
- [59] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever et al., "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [60] T. Brown, B. Mann, N. Ryder, and et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901.
- [61] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and Efficient Foundation Language Models," *arXiv e-prints*, p. arXiv:2302.13971, Feb. 2023.
- [62] A. Dubey, A. Jauhri, A. Pandey, and et al., "The Llama 3 Herd of Models," *arXiv e-prints*, p. arXiv:2407.21783, Jul. 2024.
- [63] A. Q. Jiang, A. Sablayrolles, A. Mensch, and et al., "Mistral 7B," *arXiv e-prints*, p. arXiv:2310.06825, Oct. 2023.
- [64] A. Q. Jiang, A. Sablayrolles, A. Roux, and et al., "Mixtral of Experts," *arXiv e-prints*, p. arXiv:2401.04088, Jan. 2024.
- [65] P. Agrawal, S. Antoniak, E. B. Hanna, and et al., "Pixtral 12B," *arXiv e-prints*, p. arXiv:2410.07073, Oct. 2024.
- [66] H. Jin, W. Wei, X. Wang, W. Zhang, and Y. Wu, "Rethinking learning rate tuning in the era of large language models," in *2023 IEEE 5th International Conference on Cognitive Machine Intelligence (CogMI)*, 2023, pp. 112–121.
- [67] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient Generative LLM Inference Using Phase Splitting," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. Los Alamitos, CA, USA: IEEE Computer Society, Jul. 2024, pp. 118–132. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/ISCA59077.2024.00019>
- [68] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li, Y. Lou, L. Wang, Z. Yuan, X. Li, S. Yan, G. Dai, X.-P. Zhang, Y. Dong, and Y. Wang, "A Survey on Efficient Inference for Large Language Models," *arXiv e-prints*, p. arXiv:2404.14294, Apr. 2024.

[36] E. Choi, H. He, M. Iyyer, M. Yatskar, W.-t. Yih, Y. Choi, P. Liang 和 L. Zettlemoyer, “QuAC: 上下文中的问答”, 参见 *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, E. Riloff, D. Jiang, J. Hockenmaier 和 J. Tsujii, 编辑。比利时布鲁塞尔: 计算语言学协会, 10 月至 11 月 2018 年, 第 2174–2184 页。[在线的]。可用: <https://aclanthology.org/D18-1241/> [37] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng 和 C. Potts, “学习用于情感分析的词向量”, 载于 *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, D. Lin, Y. Matsumoto 和 R. Mihalcea, 编辑。美国俄勒冈州波特兰: 计算语言学协会, 2011 年 6 月, 第 142–150 页。[在线的]。可用: <https://aclanthology.org/P11-1015/> [38] P. Rajpurkar, J. Zhang, K. Lopyrev 和 P. Liang, “SQuAD: 机器理解文本的 100,000+ 个问题”, 见 *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, J. Su, K. Duh 和 X. Carreras, 编辑。德克萨斯州奥斯汀: 计算语言学协会, 2016 年 11 月, 第 2383–2392 页。[在线的]。可用: <https://aclanthology.org/D16-1264/> [39] C.-Y. Lin 和 F. J. Och, “使用最长公共子序列和跳过二元组统计数据自动评估机器翻译质量”, *Proceedings of the 42nd Annual Meeting of the Association for Computational Linguistics (ACL-04)*, 西班牙巴塞罗那, 2004 年 7 月, 第 605–612 页。[在线的]。可用: <https://aclanthology.org/P04-1077/> [40] Y. G. au, W. Wang, D. Wang, H. Wang 和 Z. Zhang, “通信约束下的云边缘推理: 数据量化和提前退出”, *2022 International Symposium on Wireless Communication Systems (ISWCS)*, 2022 年, 第 1–6 页。[41] D. Xu, X. He, T. Su 和 Z. Wang, “云、边缘和终端设备上的神经网络分区调查”, *arXiv e-prints*, 第 11 页。arXiv:2304.10020, 2023 年 4 月。[42] A. Graves, “循环神经网络的自适应计算时间”, *arXiv e-prints*, 第 14 页。arXiv:1603.08983, 2016 年 3 月。[43] R. Schwartz, G. Stanovsky, S. Swayamdipta, J. Dodge 和 N. A. Smith, “适合这项工作的工具: 匹配模型和实例复杂性”, 载于 *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schuster 和 J. Tetreault, 编辑。在线: 计算语言学协会, 2020 年 7 月, 第 6640–6651 页。[在线的]。可用: <https://aclanthology.org/2020.acl-main.593/> [44] T. Schuster, A. Fisch, T. Jaakkola 和 R. Barzilay, “通过置信自适应变压器实现一致加速推理”, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, M.-F. Moens, X. Huang, L. Specia 和 S. W.-t. 嗯, 编辑。在线和多米尼加共和国蓬塔卡纳: 计算语言学协会, 2021 年 11 月, 第 4962–4979 页。[在线的]。可用: <https://aclanthology.org/2021.emnlp-main.406/> [45] F. Ilhan, K.-H. Chow, S. Hu, T. Huang, S. Tekin, W. Wei, Y. Wu, M. Lee, R. Kompella, H. Latapie, G. Liu 和 L. Liu, “利用 eenet 进行自适应深度神经网络推理优化”, 载于 *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2024 年 1 月, 第 1373–1382 页。[46] S. Teerapittayanon, B. McDanel 和 H. Kung, “Branchynet: 通过早期退出神经网络进行快速推理”, *2016 23rd International Conference on Pattern Recognition (ICPR)*, 2016 年, 第 2464–2469 页。[47] W. Liu, P. Zhou, Z. Wang, Z. Zhu, H. Deng 和 Q. Ju, “FastBERT: 具有自适应推理时间的自蒸馏 BERT”, 见 *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schuster 和 J. Tetreault, 编辑。在线: 计算语言学协会, 2020 年 7 月, 第 6035–6044 页。[在线的]。可用: <https://aclanthology.org/2020.acl-main.537/> [48] L. Hou, Z. Huang, L. Shang, X. Jiang, X. Chen 和 Q. Liu, “Dynabert: 具有自适应宽度和深度的动态 bert”, 见 *Proceedings of the 34th International Conference on Neural Information Processing Systems*, 序列号。NIPS '20。美国纽约州 Red Hook: Curran Associates Inc., 2020 年。[49] W. Zhou, C. Xu, T. Ge, J. McAuley, K. Xu 和 F. Wei, “Bert 失去耐心: 快速而稳健的推理并提前退出”, 见 *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan 和 H. Lin, 编辑, 卷。33。Curran Associates, Inc., 2020, 第 18330–18341 页。[50] L. Del Corro, A. Del Giorno, S. Agarwal, B. Yu, A. Awadallah 和 S. Mukherjee, “SkipDecode: 自回归跳跃解码

高效 LLM 推理的批处理和缓存”, *arXiv e-prints*, 第 arXiv:2307.02628, 2023 年 7 月。[51] S. Bae, J. Ko, H. Song 和 S.-Y. Yun, “具有同步并行解码的自回归语言模型的快速且鲁棒的早期退出框架”, 参见 *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino 和 K. Bali, 编者, 新加坡: 计算语言学协会, 2023 年 12 月, 第 5910–5924 页。[在线]: <https://aclanthology.org/2023.emnlp-main.362/> [52] S. Tang, Y. Wang, Z. Kong, T. Zhang, Y. Li, C. Ding, Y. Wang, Y. Liang 和 D. Xu, “您需要多重退出: 加速统一视觉语言模型的动态早期退出”, 载于 *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023 年 6 月, 第 10781–10791 页。[53] N. Varshney, A. Chatterjee, M. Parmar 和 C. Baral, “通过 ‘LITE’ 指令调整启用中间层解码来研究 LLaMA 推理的加速”, *Findings of the Association for Computational Linguistics: NAACL 2024*, K. Duh, H. Gomez 和 S. Bethard, 墨西哥城, 计算语言学协会, 2024 年 6 月, 第 3656–3677 页。[在线]。https://aclanthology.org/2024.findings-naacl.232 [54] X. Pan, Y. Chen, Y. Li, B. Ding 和 J. Zhou, “EE-Tuning: 用于调整早期退出大语言模型的经济且可扩展的解决方案”, p. arXiv: 2402.00518, 2024 年 2 月。[55] J. Kaddour, J. Harris, M. Mozes, H. Bradley, R. Raileanu 和 R. McHardy, “大语言模型的挑战和应用” *arXiv e-prints*, 第 arXiv: 2307.10169, 2023 年 7 月。[56] S. Hu, T. Huang, K.-H. Chow, W. Wei, Y. Wu 和 L. Liu, “Zipzap: 用于区块链大规模欺诈检测的语言模型的有效训练”, 《WWW ’24》, 美国纽约协会。计算机, 2024 年, 第 2807–2816 页。[在线]: <https://doi.org/10.1145/3589334.3645352> [57] J. Shi, A. Shirali, B. Jin, S. Zhou, W. Hu, R. Rangaraj, S. Wang, J. Han, Z. Wang, U. Lall, Y. Wu, L. Bobadilla, 和 G. Narasimhan, “天气预报的深度学习和基础模型: 一项调查”, *arXiv e-prints*, 第 arXiv: 2501.06907, 2025 年 1 月。[58] A. Radford, K. Narasimhan, T. Salimans 和 I. Sutskever, “通过生成预训练提高语言理解”, 2018 年。[59] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, “语言模型是无监督的多任务学习者”, *OpenAI blog*, 第 1 卷, 第 8 期, 第 9 页, 2019 年。[60] T. Brown, B. Mann, N. Ryder 等人, “语言模型是小样本学习者”, *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan 和 H. Lin, Eds., 第 33 卷。Curran Associates, Inc., 2020 年, 第 1877–1901 页。[61] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave 和 G. Lample, “LLaMA: 开放且高效的基础语言模型”, *arXiv e-prints*, 第 arXiv:2302.13971, 2023 年 2 月。[62] A. Dubey, A. Jauhri, A. Pandey 等人, “The Llama 3 Herd of Models”, *arXiv e-prints*, p. arXiv:2407.21783, 2024 年 7 月。[63] A. Q. Jiang, A. Sablayrolles, A. Mensch 等人, “Mistral 7B”, *arXiv e-prints*, p. arXiv:2310.06825, 2023 年 10 月。[64] A. Q. Jiang, A. Sablayrolles, A. Ro ux 等, “Mixtral of Experts”, *arXiv e-prints*, p. arXiv:2401.04088, 2024 年 1 月。[65] P. Agrawal, S. Antoniak, E. B. Hanna 等 al., “Pixtral 12B”, *arXiv e-prints*, p. arXiv: 2410.07073, 2024 年 10 月。[66] H. Jin, W. Wei, X. Wang, W. Zhang 和 Y. Wu, “重新思考大语言模型时代的学习率调整”, *2023 IEEE 5th International Conference on Cognitive Machine Intelligence (CogMI)*, 2023 年, 第 177 页。112–121. P. Patel, E. Choukse, C. Zhu, A. Shah, I. G. oiri, S. Maleki 和 R. Bianchini, “Splitwise: 使用相分裂的高效生成 LLM 推理”, *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 美国加利福尼亚州洛杉矶阿拉米托斯: IEEE 计算机协会, 2024 年 7 月, 第 112 页。118–132。可用: <https://doi.ieeeecomputersociety.org/10.1109/ISCA59077.2024.00019> [68] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li, Y. Lou, L. Wang, Z. Yuan, X. Li, S. Yan, G. Dai, X.-P. Y. Dong 和 Y. Wang, “大语言模型的高效推理调查”, *arXiv e-prints*, arXiv: 2404.14294, 2024 年 4 月。